

Quick Study Guide: Cloud Computing, Virtualization, and Containers

Purpose of this guide

This guide is designed to help understand *core concepts, mechanisms, and trade-offs* in modern cloud platforms. It emphasizes **how systems work** and **why design choices matter**.

1. Cloud Computing Fundamentals

1.1 Cloud Stack–Layer Model

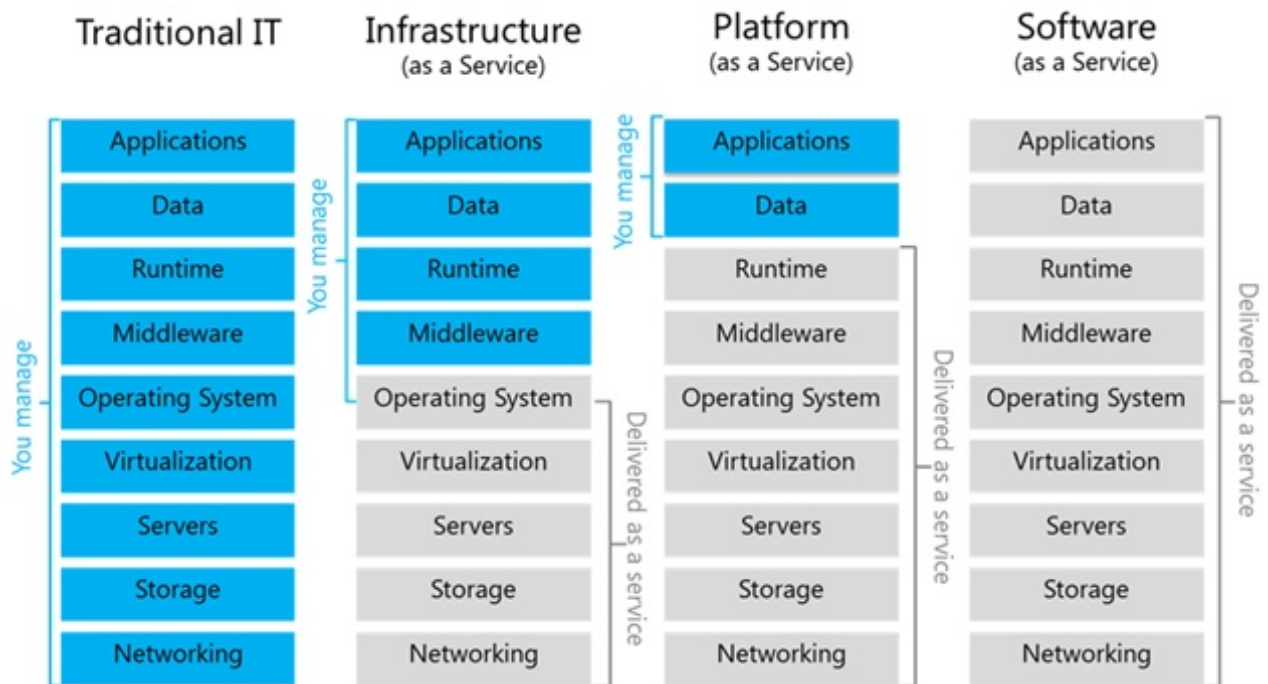
Cloud services are best understood as **layers of responsibility**:

- **Physical Infrastructure:** Servers, storage, networking
- **Virtualization Layer:** Hypervisors, VMs, virtual networks
- **Operating System Layer**
- **Runtime & Middleware**
- **Applications**

Service Models

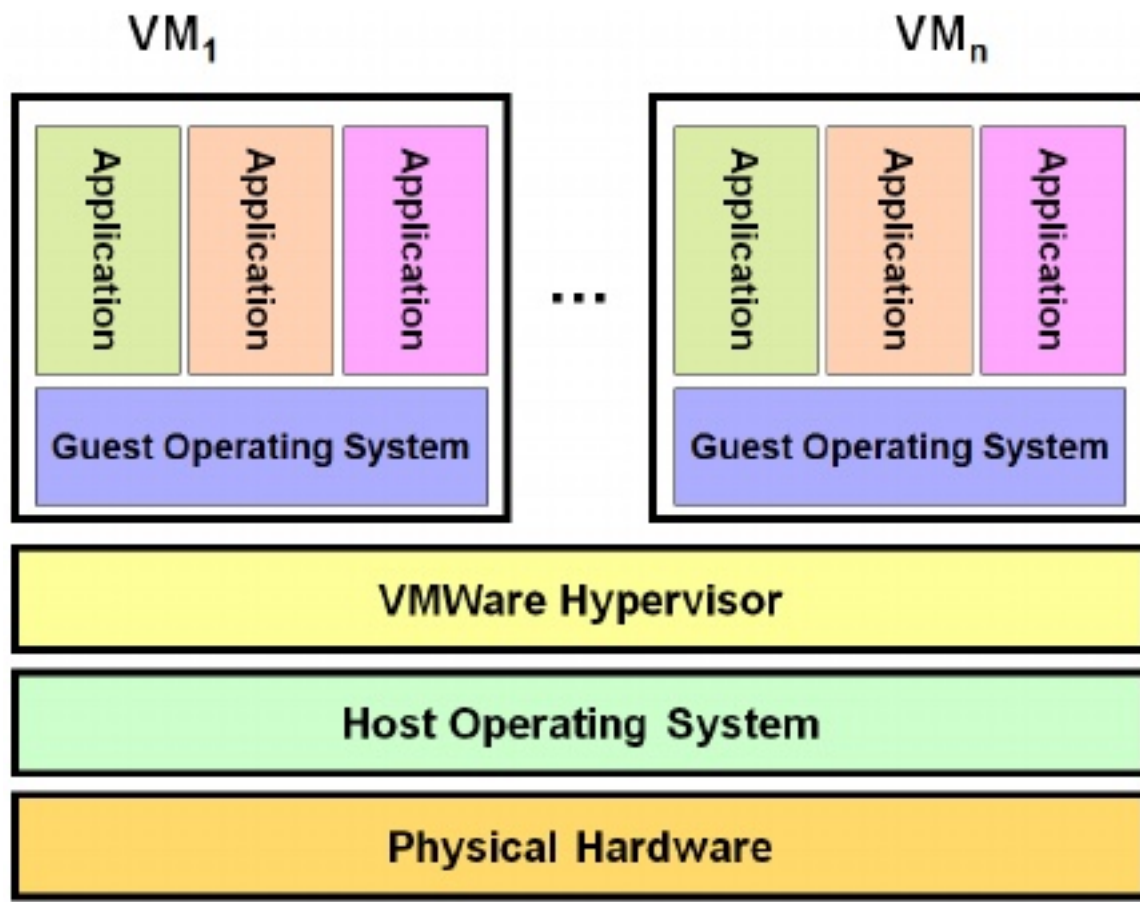
Model	User Controls	Provider Controls
IaaS	OS, middleware, apps	Hardware, virtualization
PaaS	Applications	OS, runtime, infrastructure
SaaS	Configuration & usage	Entire stack

 **Key insight:** As you move from IaaS → SaaS, *control decreases but operational simplicity increases*.

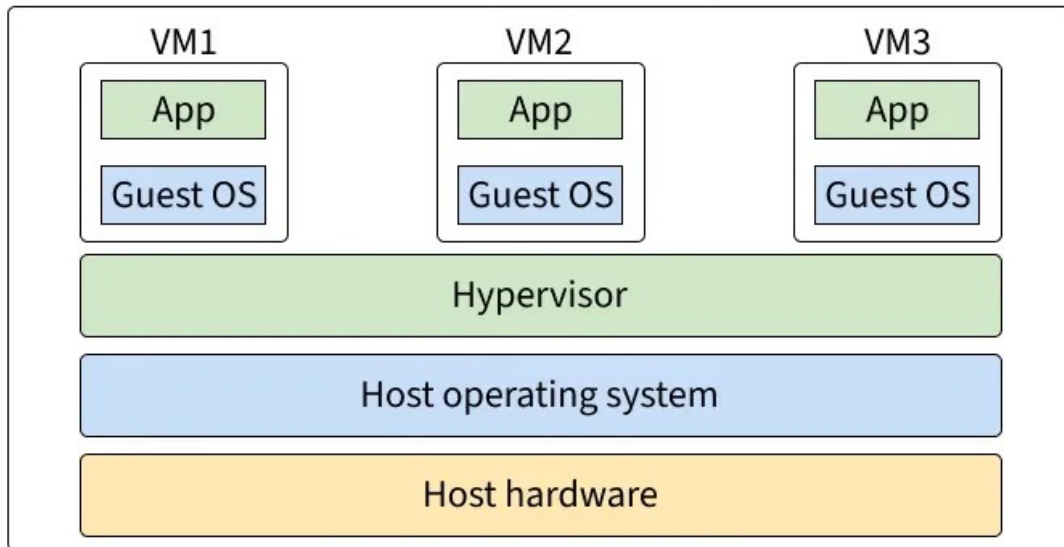


2. Virtualization Architecture

2.1 Core Components of a VM System



Virtual machines



- **Host Operating System**
 - Manages physical hardware (in hosted setups)
 - Provides device drivers and scheduling
- **Hypervisor (Virtual Machine Monitor)**
 - Creates, runs, and isolates VMs
 - Intercepts privileged instructions
- **Guest Operating System**
 - Runs inside a VM
 - Believes it has exclusive access to hardware

 **Key idea:** The hypervisor is the *trust boundary* of the VM system.

2.2 Hypervisor Types

Type	Description	Characteristics
Type 1 (Bare-metal)	Runs directly on hardware	High performance, strong isolation
Type 2 (Hosted)	Runs on a host OS	Easier to use, more overhead

 **Design implication:** Production clouds prefer Type 1 hypervisors for efficiency and security.

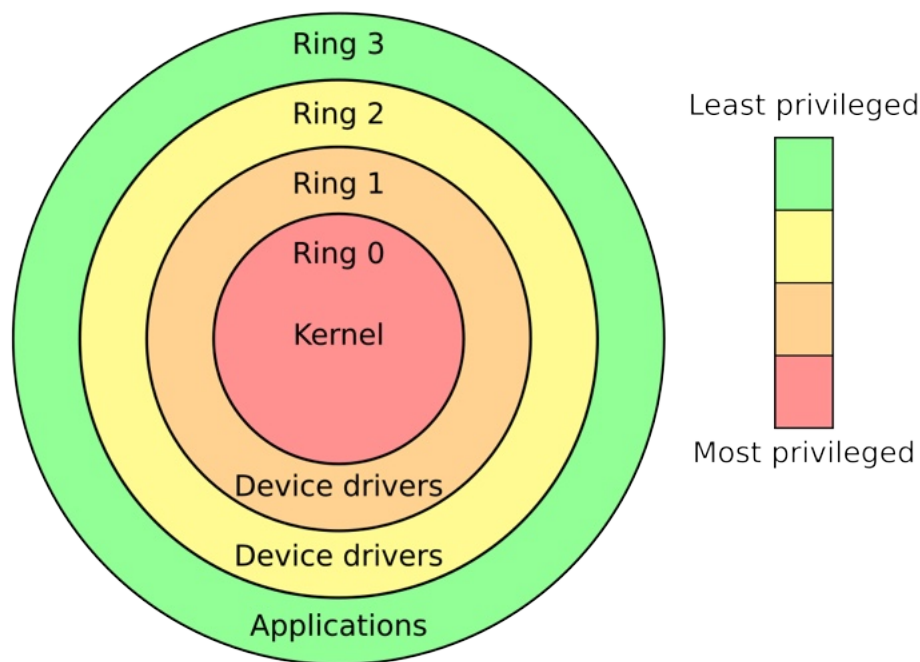
2.3 CPU Privilege and Instruction Handling

- Modern CPUs use **privilege rings**
- **Ring 0** = highest privilege (kernel / hypervisor)
- User applications run in lower rings

Trap-and-emulate model:

1. Guest OS executes privileged instruction
2. CPU traps to hypervisor
3. Hypervisor emulates or denies the instruction

 **Why it matters:** Safe virtualization depends on controlling privileged operations.



2.4 Virtualization Techniques

Technique	Guest OS Modification	Performance
Full Virtualization	No	Moderate
Para-Virtualization	Yes	Often higher
Hardware-assisted	No	High

 **Conceptual takeaway:** Reducing traps improves performance, but may require guest cooperation.

2.5 VM Isolation & Resource Management

Important virtualization properties:

- **Isolation:** One VM cannot affect another
- **Encapsulation:** VM exists as a set of files
- **Partitioning:** Hardware is divided among VMs

📌 **Why isolation matters:** Fault containment and multi-tenant security.

3. Cost and Availability Concepts

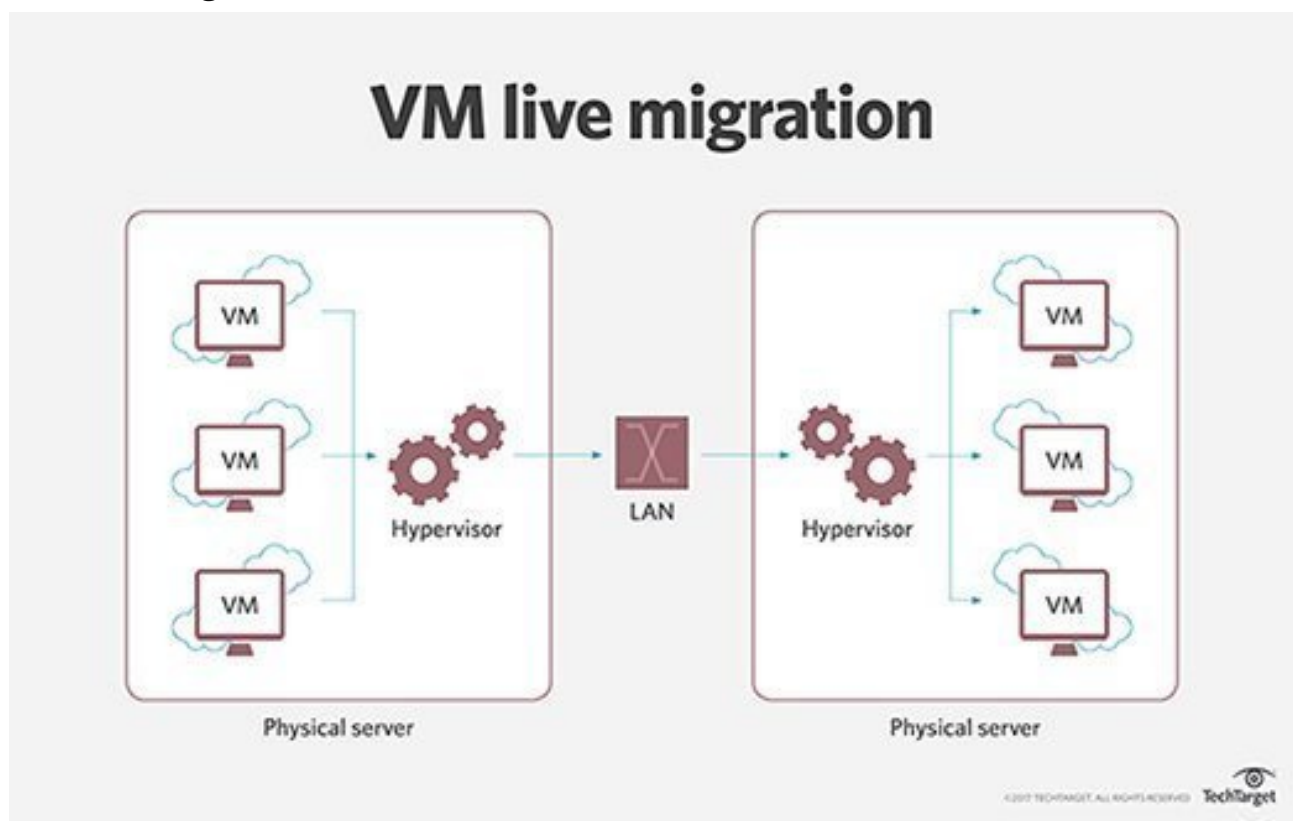
3.1 Total Cost of Ownership (TCO)

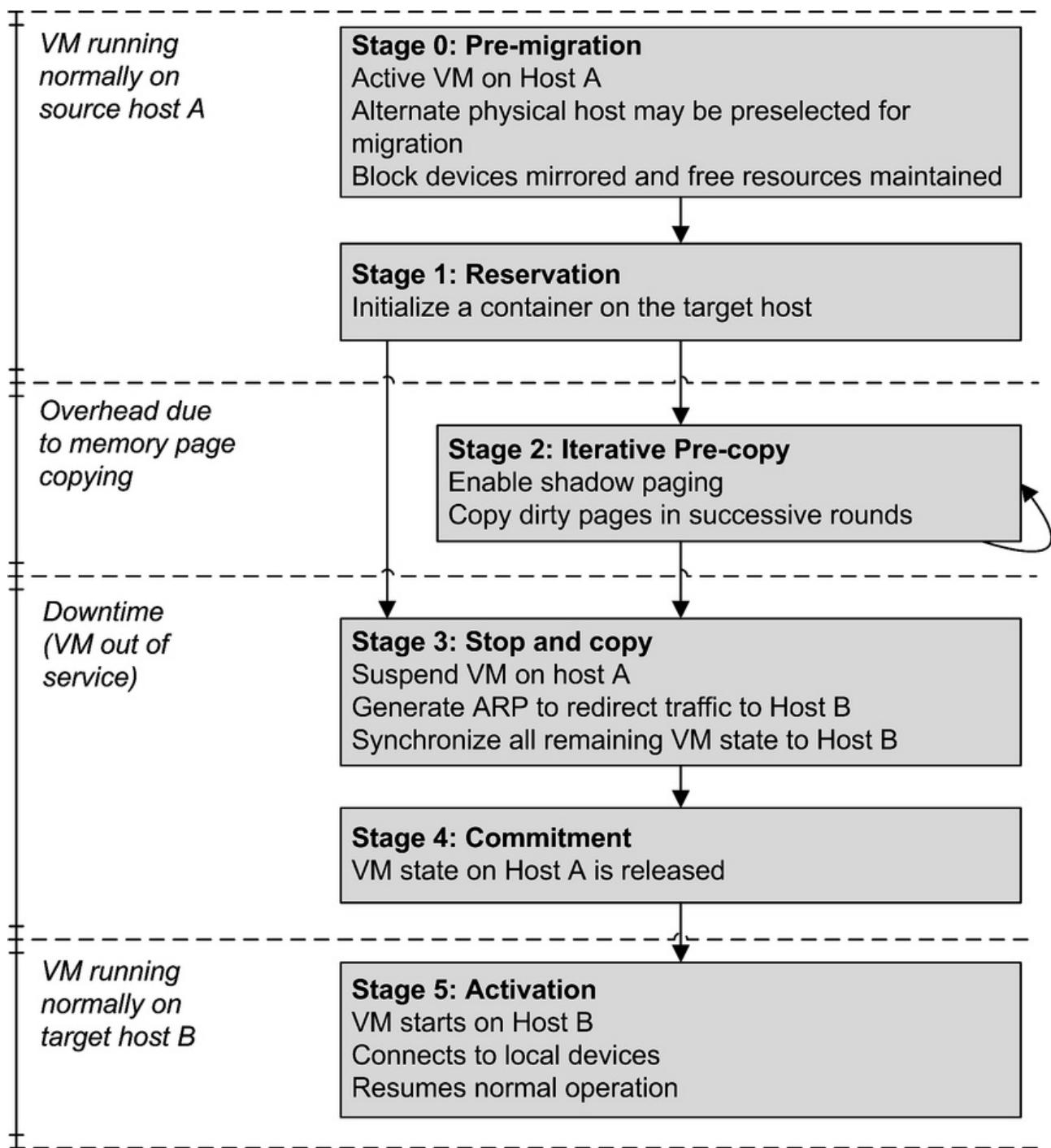
TCO combines:

- **Capital Expenditure (CapEx):** Hardware purchase
- **Operational Expenditure (OpEx):** Power, cooling, admin labor

📌 **Cloud value proposition:** Shift from CapEx-heavy to OpEx-optimized models.

3.2 Live Migration





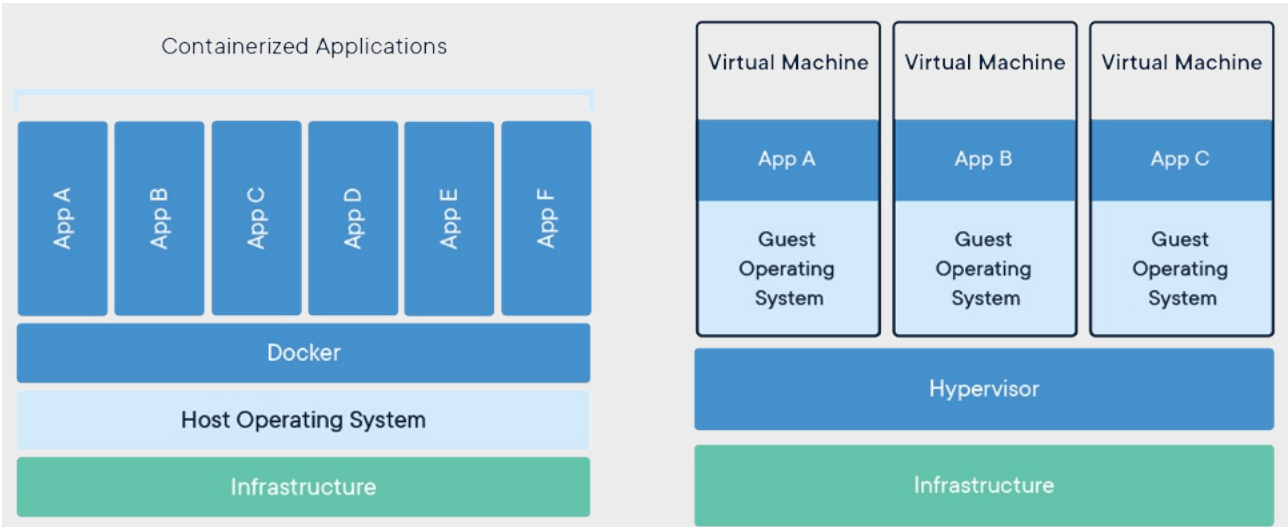
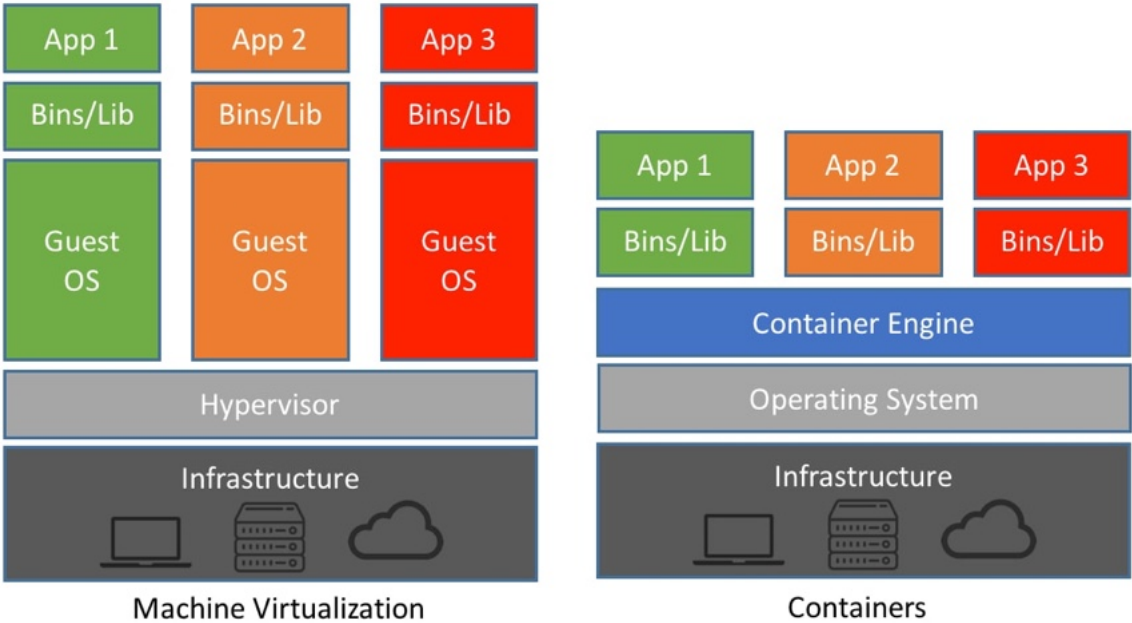
Live migration allows moving a running VM between hosts with minimal downtime.

- **Pre-copy migration**
 - Memory copied while VM runs
 - Short final pause
 - Safer for user-facing services
- **Post-copy migration**
 - VM resumes immediately on target
 - Pages fetched on demand
 - Faster, but riskier

 **Trade-off:** Reliability vs migration speed.

4. Container Technology

4.1 Containers vs Virtual Machines



Feature	VM	Container
Kernel	Separate	Shared
Startup Time	Slow	Fast
Isolation	Strong	Lightweight
Resource Overhead	High	Low

📌 **Key insight:** Containers trade isolation for speed and density.

4.2 Linux Kernel Mechanisms Behind Containers

- **Namespaces**
 - Process isolation
 - Network isolation
 - Filesystem isolation
- **Control Groups (cgroups)**
 - CPU, memory, I/O limits

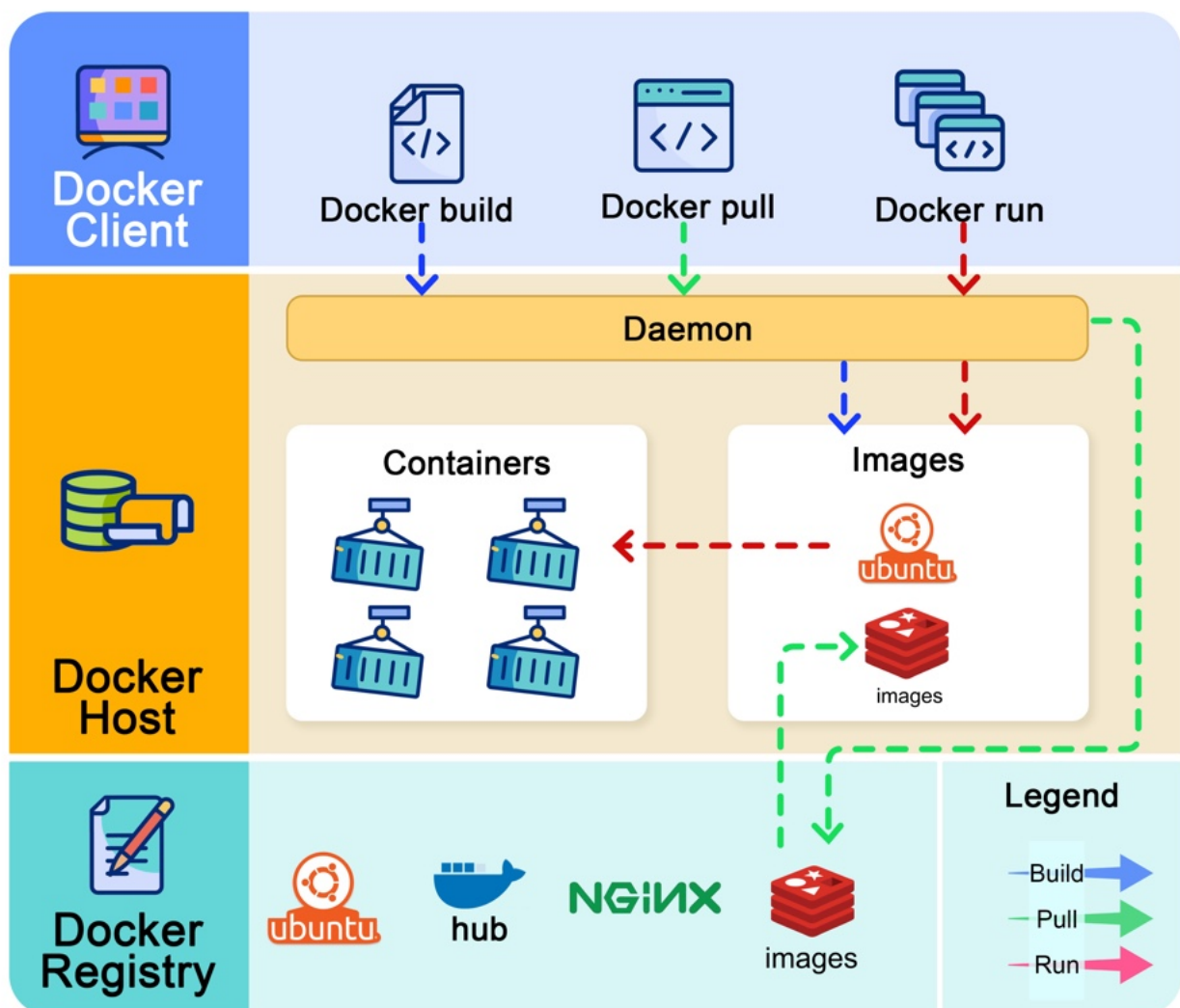
📌 **Mental model:**

Namespaces define *what you can see*
cgroups define *what you can use*

4.3 Docker Architecture

How does Docker Work ?

 blog.bytebytego.com





DOCKER IMAGE LAYERS

www.learnitguide.net

1. Started Image Layer 3 as a container and accessible by users

1. Started Image Layer v1 as a container.
2. Installed and Configured https web server.
3. Committed new layer v2

1. Started Base Image (**docker.io/centos**) as a container.
2. Package Updated on Base Image using "yum update".
3. Committed new layer v1

Pulled CentOS image from Docker Hub using docker pull command. **Repo: docker.io/centos**



Core components:

- Docker Client
- Docker Daemon
- Docker Image
- Docker Container
- Image Registry

Image → Container lifecycle:

1. Image pulled from registry
2. Read-only layers mounted
3. Writable layer added
4. Process starts

4.4 Docker Storage: Copy-on-Write

- Images are immutable
- Containers write only to a private layer
- Shared layers reduce disk usage

 **Why it matters:** Efficient storage and rapid deployment.

4.5 Container Networking

- Default bridge network:
 - Private IP per container
 - Internal communication allowed
- **Port publishing:**
 - Required to expose container services externally

 **Security benefit:** Services are isolated unless explicitly exposed.

5. Orchestration Concepts

5.1 Pods (Kubernetes)

- A **Pod** is a group of containers that:
 - Share networking
 - Share storage volumes
- Used when containers are tightly coupled

 **Design principle:** One Pod = one logical application unit.

6. Systems Design Thinking (Applied Knowledge)

Combining VMs and Containers

Best-practice architectures often:

- Use **VMs** for:
 - Strong isolation
 - Stateful workloads
 - Hardware-specific workloads
- Use **Containers** for:
 - Microservices
 - Frontend and API layers
 - Rapid scaling

 **Key trade-off:**

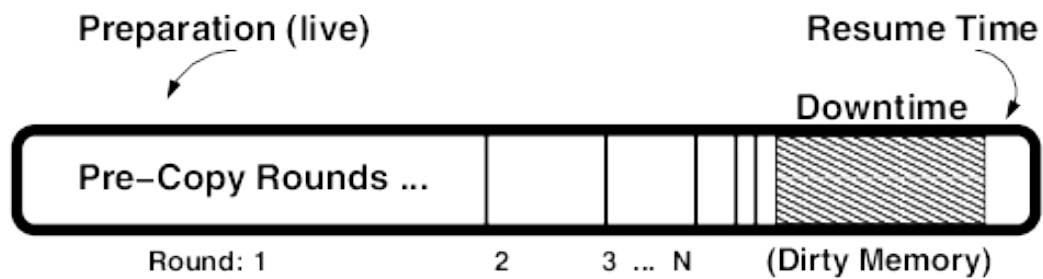
- VMs → security & isolation
- Containers → agility & efficiency

6. Architecture Strategy (What They Expect)

- **VMs:**
 - Strong isolation
 - Stateful services (databases, ML inference with GPUs)
- **Containers:**
 - LMS, SIS, admin dashboards
 - Microservices, scalability, CI/CD

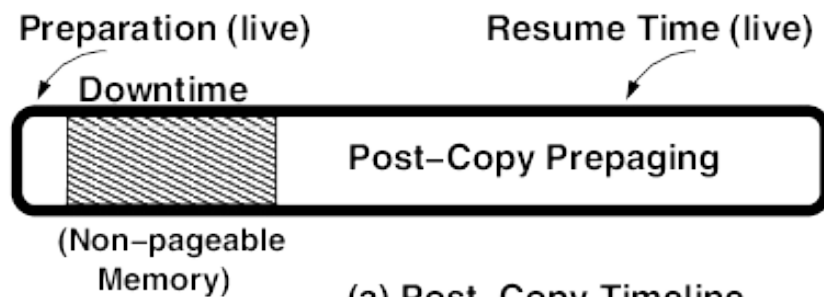
Live Migration Choice (LMS/SIS)

- **Pre-copy:**
 - Preferred when uptime is critical
 - Minimal service disruption
- **Post-copy:**
 - Faster migration but riskier
 - Temporary performance degradation



(a) Pre-Copy Timeline

----- Time ----->



(a) Post-Copy Timeline

Rationale

- **VMs + Containers:**
 - Best balance of security, performance, and agility
- **All-VM:**
 - Strong isolation, poor scalability, higher cost
- **All-Containers:**
 - Fast and lightweight, weaker isolation, harder state management

CET 322 Question Bank (Revision):

MCQ 1 — Cloud service models

Which cloud service model gives users the **most control** over the software stack?

- A. Software as a Service
- B. Platform as a Service
- C. Infrastructure as a Service
- D. Function as a Service

Correct answer: C

Explanation: IaaS leaves OS, middleware, and applications under user control, unlike PaaS or SaaS.

MCQ 2 — SaaS characteristics

Which activity is the user **primarily responsible for** when using a SaaS application?

- A. Patching the operating system
- B. Managing application servers
- C. Configuring user-level settings
- D. Maintaining virtualization software

Correct answer: C

Explanation: In SaaS, the provider manages infrastructure and software; users mainly configure and use the app.

MCQ 3 — PaaS responsibility

In a PaaS environment, which component is typically managed by the **cloud provider**?

- A. Application data
- B. Application code
- C. Runtime and operating system
- D. User access policies

Correct answer: C

Explanation: PaaS abstracts OS and runtime, allowing developers to focus on code and data.

MCQ 4 — Deployment models

A cloud environment dedicated to a single organization and not shared publicly is best described as:

- A. Public cloud
- B. Community cloud
- C. Hybrid cloud
- D. Private cloud

Correct answer: D

Explanation: Private clouds are dedicated to one organization, regardless of location.

MCQ 5 — Hybrid cloud definition

Which feature is essential for a system to be considered **hybrid cloud**?

- A. Use of virtualization
- B. Combination of VMs and containers
- C. Integration between different environments
- D. Presence of multiple service models

Correct answer: C

Explanation: Hybrid cloud requires integrated environments, not just coexistence.

MCQ 6 — Virtualization purpose

What is the primary goal of virtualization in cloud computing?

- A. Increasing clock speed
- B. Eliminating operating systems
- C. Sharing physical resources efficiently
- D. Removing the need for networking

Correct answer: C

Explanation: Virtualization allows safe, efficient sharing of hardware among workloads.

MCQ 7 — Hypervisor role

Which component is responsible for **creating and managing virtual machines**?

- A. Guest operating system
- B. Host operating system

- C. Hypervisor (VMM)
- D. Container runtime

Correct answer: C

Explanation: The hypervisor allocates resources and enforces isolation between VMs.

MCQ 8 — Type 1 vs Type 2 hypervisors

A hypervisor that runs **directly on hardware** is classified as:

- A. Hosted hypervisor
- B. Type 2 hypervisor
- C. Guest hypervisor
- D. Type 1 hypervisor

Correct answer: D

Explanation: Type 1 (bare-metal) hypervisors run directly on physical hardware.

MCQ 9 — Virtualization overhead

Which factor explains why virtual machines may perform slightly worse than bare metal?

- A. Kernel sharing
- B. Hypervisor mediation
- C. Lack of isolation
- D. Absence of scheduling

Correct answer: B

Explanation: The hypervisor introduces overhead when managing CPU, memory, and I/O.

MCQ 10 — Containers vs VMs

Which statement best describes containers compared to virtual machines?

- A. Containers include a full guest OS
- B. Containers virtualize hardware
- C. Containers share the host OS kernel
- D. Containers require a hypervisor

Correct answer: C

Explanation: Containers rely on kernel sharing, making them lightweight.

MCQ 11 — Docker's role

Docker is best described as:

- A. A cloud provider
- B. A hypervisor
- C. A container management platform
- D. A virtual machine monitor

Correct answer: C

Explanation: Docker simplifies building, running, and distributing containers.

MCQ 12 — Docker images

What best describes a Docker image?

- A. A running container
- B. A writable filesystem
- C. A template used to create containers
- D. A physical disk partition

Correct answer: C

Explanation: Images are read-only templates used to instantiate containers.

MCQ 13 — Image layering

Why are Docker image layers useful?

- A. They increase isolation
- B. They reduce storage duplication
- C. They remove the need for registries
- D. They eliminate networking

Correct answer: B

Explanation: Shared layers reduce disk usage and speed up builds.

MCQ 14 — Registries

What is the main purpose of a container registry?

- A. Running containers
- B. Scheduling workloads
- C. Storing and distributing images
- D. Managing user identities

Correct answer: C

Explanation: Registries host images for distribution and reuse.

MCQ 15 — Namespace function

In containers, namespaces are primarily used to:

- A. Limit resource usage
- B. Encrypt data
- C. Isolate system views
- D. Improve CPU speed

Correct answer: C

Explanation: Namespaces isolate what a container can see (processes, network, etc.).

MCQ 16 — cgroups function

Control groups (cgroups) are responsible for:

- A. Network isolation
- B. Filesystem layering
- C. Resource limiting and accounting
- D. Image versioning

Correct answer: C

Explanation: cgroups manage how much CPU, memory, and I/O a container can use.

MCQ 17 — Overlay filesystem

Overlay-style filesystems are mainly used to:

- A. Encrypt container data
- B. Merge image layers efficiently
- C. Improve CPU scheduling
- D. Replace virtual disks

Correct answer: B

Explanation: Overlay filesystems combine read-only layers with a writable layer.

MCQ 18 — Container networking

Why is port mapping needed in container networking?

- A. To assign IP addresses
- B. To expose container services externally
- C. To isolate containers
- D. To manage CPU usage

Correct answer: B

Explanation: Port mapping connects container ports to host/network ports.

MCQ 19 — Bridge network

Which Docker network mode provides isolation using a virtual switch?

- A. Host
- B. None
- C. Bridge
- D. Overlay

Correct answer: C

Explanation: Bridge networking uses a virtual bridge to connect containers.

MCQ 20 — Host networking

Which trade-off is most associated with host networking mode?

- A. Higher isolation
- B. Increased startup time
- C. Reduced security isolation
- D. No external connectivity

Correct answer: C

Explanation: Host networking removes network isolation.

MCQ 21 — Container lifecycle

Which action permanently frees container resources?

- A. Pause
- B. Stop
- C. Restart
- D. Remove

Correct answer: D

Explanation: Stopped containers still exist; removal deletes them.

MCQ 22 — Orchestration need

Why is container orchestration important in production?

- A. Containers cannot run without it
- B. Manual management does not scale
- C. It replaces virtualization
- D. It eliminates failures

Correct answer: B

Explanation: Orchestration automates management at scale.

MCQ 23 — Kubernetes purpose

Kubernetes is primarily designed to:

- A. Build container images
- B. Replace Docker
- C. Orchestrate containerized workloads
- D. Manage physical servers

Correct answer: C

Explanation: Kubernetes automates deployment, scaling, and recovery.

MCQ 24 — Pods

In Kubernetes, the smallest deployable unit is a:

- A. Container
- B. Node
- C. Service
- D. Pod

Correct answer: D

Explanation: Kubernetes schedules pods, not individual containers.

MCQ 25 — Pod characteristics

Containers in the same pod always share:

- A. File systems
- B. IP address and network namespace
- C. CPU limits
- D. Storage volume types

Correct answer: B

Explanation: All containers in a pod share networking.

MCQ 26—Scaling

Scaling an application in Kubernetes usually means:

- A. Adding more nodes
- B. Increasing CPU frequency
- C. Increasing pod replicas
- D. Rebuilding images

Correct answer: C

Explanation: Scaling commonly adjusts the number of replicas.

MCQ 27—Self-healing

What happens when a pod fails in Kubernetes?

- A. Nothing until user intervention
- B. The entire cluster restarts
- C. Kubernetes replaces the pod automatically
- D. Data is deleted

Correct answer: C

Explanation: Kubernetes continuously enforces desired state.

MCQ 28—Rolling updates

What is the main benefit of rolling updates?

- A. Faster builds
- B. Zero-downtime application updates
- C. Stronger security
- D. Lower storage usage

Correct answer: B

Explanation: Rolling updates replace pods gradually.

MCQ 29 — Stateless apps

Which property best describes a stateless application?

- A. Stores local data
- B. Difficult to scale
- C. Can be replaced without data loss
- D. Requires persistent volumes

Correct answer: C

Explanation: Stateless apps keep no local state.

MCQ 30 — Elasticity

Elasticity in cloud computing refers to:

- A. Performance tuning
- B. Static provisioning
- C. Dynamic resource adjustment
- D. Security hardening

Correct answer: C

Explanation: Elasticity adapts resources to demand.

MCQ 31 — Pay-as-you-go

Which cloud cost model charges based on actual usage?

- A. Fixed subscription
- B. Capital investment
- C. Pay-as-you-go
- D. Reserved hardware

Correct answer: C

Explanation: Usage-based billing aligns cost with demand.

MCQ 32 — TCO

Total Cost of Ownership includes:

- A. Hardware cost only
- B. Initial purchase only
- C. Long-term operational costs
- D. Licensing fees only

Correct answer: C

Explanation: TCO covers all costs over time.

Definitions

SA1 — Cloud service models

Question:

Explain the main differences between IaaS, PaaS, and SaaS in terms of user responsibility.

Model answer:

In **Infrastructure as a Service (IaaS)**, the cloud provider manages the physical infrastructure such as servers, storage, networking, and virtualization, while the user is responsible for the operating system, middleware, applications, and data.

In **Platform as a Service (PaaS)**, the provider also manages the operating system and runtime environment, allowing the user to focus mainly on developing and deploying applications and managing data.

In **Software as a Service (SaaS)**, the provider delivers a complete application and manages almost everything, while the user mainly configures settings and uses the software.

SA2 — Deployment models

Question:

Describe public, private, and hybrid cloud deployment models and give one reason to choose each.

Model answer:

A **public cloud** is shared among multiple customers and operated by a cloud provider, making it suitable for cost efficiency and scalability.

A **private cloud** is dedicated to a single organization, offering greater control and compliance.

A **hybrid cloud** integrates private and public environments, allowing organizations to keep sensitive workloads private while using public cloud resources for scalability or cost savings.

SA3 — Virtualization purpose

Question:

Why is virtualization considered a foundational technology for cloud computing?

Model answer:

Virtualization allows multiple isolated virtual machines to run on the same physical hardware, improving resource utilization. It enables rapid provisioning, isolation between users, and flexible resource allocation, which are essential for scalability, elasticity, and multi-tenancy in cloud computing.

SA4 — Hypervisors

Question:

Explain the difference between Type 1 and Type 2 hypervisors and where each is typically used.

Model answer:

A **Type 1 hypervisor** runs directly on physical hardware and is commonly used in data centers and cloud environments due to better performance and isolation.

A **Type 2 hypervisor** runs on top of a host operating system and is typically used for development, testing, or personal use.

SA5 — Containers vs virtual machines

Question:

Compare containers and virtual machines with respect to isolation, overhead, and startup time.

Model answer:

Virtual machines provide strong isolation because each VM includes a full guest operating system, but they have higher overhead and slower startup times.

Containers share the host OS kernel, making them lightweight and fast to start, but they offer slightly weaker isolation compared to VMs.

SA6 — Docker images

Question:

What is a container image and how does layering improve efficiency?

Model answer:

A container image is a read-only template that contains an application and its dependencies. Layering allows images to share common components, reducing storage usage and speeding up image builds and transfers.

SA7 — Namespaces and cgroups

Question:

Explain how namespaces and cgroups work together to support container isolation.

Model answer:

Namespaces isolate what a container can see, such as processes, networking, and filesystems.

Cgroups limit and track how much CPU, memory, and I/O a container can use.

Together, they provide both isolation and fair resource usage.

SA8 — Container networking

Question:

Why is port mapping necessary in containerized applications?

Model answer:

Containers run in isolated network environments with their own internal ports. Port mapping allows traffic from the host or external network to reach services running inside a container.

SA9 — Orchestration

Question:

Why is container orchestration essential for production environments?

Model answer:

Orchestration automates deployment, scaling, networking, and failure recovery for large numbers of containers. Without orchestration, managing containers manually becomes inefficient and error-prone at scale.

SA10 — Kubernetes pods

Question:

What is a Kubernetes pod and why does Kubernetes schedule pods instead of containers?

Model answer:

A pod is the smallest deployable unit in Kubernetes and may contain one or more containers that share networking and storage. Kubernetes schedules pods so tightly coupled containers can run together as a single unit.

SA11 — Kubernetes services

Question:

Explain why services are required in Kubernetes even when pods are running correctly.

Model answer:

Pods are temporary and can be replaced at any time, causing IP changes. Services provide a stable endpoint and load balancing so clients can reliably access applications.

SA12 — Scaling vs availability

Question:

Differentiate between scaling and high availability in cloud systems.

Model answer:

Scaling increases or decreases resources to handle workload demand, while high availability focuses on keeping services running despite failures. Scaling does not guarantee availability, and availability does not always involve scaling.

SA13 — Persistent storage**Question:**

Why does Kubernetes separate storage from pod lifecycles?

Model answer:

Because pods are ephemeral, separating storage ensures data is preserved even if pods restart or are replaced. This allows stateful applications to function reliably.

SA14 — Migration strategies**Question:**

Compare lift-and-shift migration with refactoring-based migration.

Model answer:

Lift-and-shift moves applications to the cloud with minimal changes, allowing fast migration but limited cloud optimization.

Refactoring redesigns applications to use cloud-native features, requiring more effort but offering better scalability and efficiency.

SA15 — Shared responsibility model**Question:**

Explain the shared responsibility model in cloud security and how it changes across service models.

Model answer:

Security responsibilities are shared between the provider and the user. In IaaS, users manage OS and applications; in PaaS, the provider manages the platform; in SaaS, the provider manages almost everything except data and access control.

SA16 — Vendor lock-in**Question:**

What is vendor lock-in and how can containerization help mitigate it?

Model answer:

Vendor lock-in occurs when applications depend heavily on one provider's proprietary services. Containers improve portability by packaging applications in a standard format that can run across environments.

Design Questions (4)

Hints:

VM vs container architecture

Solution hints:

Given the legacy nature of the application and limited refactoring budget, using **virtual machines** is the most suitable option. VMs allow the application to run with minimal changes while still benefiting from cloud infrastructure. Containers could be considered later if refactoring becomes possible.

Kubernetes deployment

Solution hints:

The application should be deployed using a **Kubernetes Deployment** with multiple replicas. A **Service** should expose the API and distribute traffic. Autoscaling can be enabled to handle traffic spikes efficiently while keeping costs under control.

Migration strategy

Solution hints:

A migration is appropriate because it minimizes risk and requires little cloud expertise. The application can be moved quickly, and optimization can be planned for later stages.

Security responsibility

Solution hints:

The cloud provider should secure the infrastructure and platform, while the organization remains responsible for data protection, access control, and application-level security. Strong IAM policies and encryption should be applied to protect customer data and meet compliance requirements.

Include enough technical details, design architecture layers, operating system stacks, and system diagrams to round out your solution.